Jenkins Content Index

1_Jenkins_intro.md

- Jenkins Overview
- How to Install Jenkins on Windows
- Configuring Jenkins on Windows
- How to Start/Stop or restart Jenkins on Windows

2_Jenkin_plugins.md

- Jenkins Plugins
- Plugin Installation
- User Management in Jenkins
- Authorization Strategies
- Jenkins Jobs/Projects
  - Overview Jobs/Projects
  - Freestyle Job/Project
- Configuring Tools in Jenkins

3_Jenkins_intro_2.md

- Jenkins Folder Structure
- What If a Jenkins User Forgets Their Password?
- What If a Jenkins Admin Forgets Their Password?
- Ways to Trigger Jobs
- How to Change the Default Port Number of Jenkins

4_basic_pipeline.md

- Pipeline Projects Overview
  - Why To Use Pipeline Projects?
  - Creating a Simple "Hello World" Pipeline Job

- Rebuild vs. Replay
- Using the Snippet Generator in Jenkins Pipeline
- Pipeline Execution Mode
- Using Environment Variables in Jenkins Pipeline
- How to Parameterize a Jenkins Job

5_Jenkin_ssh_pat.md

- How to clone git private repo with SSH keys and PAT in job/pipeline
- Personal access tokens (PAT) configuration and usage
- Jenkins Shared Libraries

6_jenkins_webhooks_backup.md

- Using GitHub Webhook Actions in Jenkins
- Backup/Migration of Jenkins Jobs

Jenkins Overview

- **Jenkins** is an open-source automation server primarily used for continuous integration (CI) and continuous delivery (CD) in software deployment.
- It automates several stages of the software development lifecycle, including:
  - **Building**
  - **Testing**
  - **Deploying** applications

Why "Continuous Delivery" is More Accurate than "Continuous Deployment"

- **Continuous Delivery (CD)**: Jenkins automates the delivery pipeline up to the point of production, but **manual intervention** (such as approval) is usually required before deploying to production.
- **Continuous Deployment (CD)**: Involves full automation, where code is deployed directly to production without manual approval.
- **Context**: Jenkins focuses on **Continuous Delivery**, as it automates much of the process but still allows for manual approval in some steps before production deployment.

Popular CI/CD Tools

- **Jenkins**
- **GitLab CI/CD**
- **TeamCity**
- **Travis CI**
- **AWS CodePipeline**
- **Azure DevOps**
- **Octopus**

Why Jenkins?

1. **Improves Quality**:
   - Automates unit tests and other testing processes.
2. **Increases Productivity**:
   - Saves time by automating repetitive tasks like builds and deployments.
3. **Reduces Risk**:

    - Minimizes human error by automating manual processes.
4. **Features**:
    - **Easy Installation**: Jenkins is simple to install and configure.
    - **Upgrades Available**: Easy to upgrade with minimal downtime.
    - **Lightweight Container Support**: Ideal for containerized environments.
    - **Advanced Security**: Offers enhanced security features.
    - **Optimized Performance**: Ensures fast performance even under load.
    - **Distributed Team Management**: Allows teams to manage Jenkins in a distributed way.
5. **Additional Features**:
    - **Open-Source**: Free and actively maintained.
    - **Good Plugin Support**: Wide variety of plugins for various integrations.
    - **Strong Community Support**: Large community for troubleshooting and best practices.
    - **Fast and Reliable**: Handles large-scale builds and deployments smoothly.
    - **Good OS Support**: Runs on various operating systems.
    - **Scripted Builds**: Customizable build scripts for complex workflows.

What is CI/CD?

- **Continuous Integration (CI)**: Frequent integration of code into a shared repository with automated builds and tests.
- **Continuous Delivery (CD)**: Ensures that code is always in a deployable state, with automated deployment and testing but requires **manual approval** for production.
- **Continuous Deployment (CD)**: Fully automates testing and deployment to production, with no manual intervention required.

Plugin Management in Jenkins

- **Tabs in Plugin Management**:
    - **Update**: Shows installed plugins that have updates available.
    - **Available**: Lists plugins available for installation.
    - **Install**: Displays plugins that are installed and do not need updates.
    - **Advanced**: Allows configuration of HTTP proxy, uploading plugins, and specifying plugin sources.

- **Important Note**: In real-world environments, plugin installation may face challenges due to proxy settings, VPNs, or network restrictions. It is advisable to configure the **HTTP Proxy** in the **Advanced** tab of Jenkins' plugin management to ensure seamless installation.

How to Install Jenkins on Windows

Prerequisites:

- **Minimum hardware requirements**:
  - 256 MB of RAM
  - 1 GB of drive space (10 GB recommended for Docker containers)
- **Recommended hardware for small teams**:
  - 4 GB+ of RAM
  - 50 GB+ of drive space

Download Location for Windows:

- Jenkins Download Page

Installation Steps:

1. Follow the installation guide for Windows.
2. For **Step 3**, select the option: **Run Jenkins as a local system service**.

Configuring Jenkins on Windows

1. **Unblocking Jenkins**:
   - Access Jenkins at: `http://localhost:8080`
   - Retrieve the **initial Administrator password** located in the Jenkins installation directory.
     - For the default location (`C:\Program Files\Jenkins`), the password is stored in the file `C:\Program Files\Jenkins\secrets\initialAdminPassword` .
   - Unlock Jenkins Documentation
2. **Customizing Jenkins with Plugins**:
   - Choose one of the following:
     - **Install Suggested Plugins**: Installs the most commonly used plugins based on typical use cases.

        - **Select Plugins to Install**: Allows manual selection of plugins to install.

3. **Create the First Administrator User**:
    - Fill out the required fields for the administrator user and click **Save and Finish**.

How to Start or Stop Jenkins on Windows

1. Open **Services** and search for **Jenkins**.
2. Select **Jenkins** from the list and choose to **Start** or **Stop** the server.

Restarting Jenkins via URL

- **Force Restart (not recommended during active builds)**:
    - Use: `http://[Jenkins URL]/restart`
- **Safe Restart (recommended)**:
    - Allows running jobs to complete before restarting: `http://[Jenkins URL]/safeRestart`

Jenkins Restart Banner

- The **Jenkins Restart Banner** informs users that Jenkins will restart, typically due to system changes, plugin updates, or upgrades. This banner helps users be aware of ongoing system changes and prevents disruption to ongoing tasks.

Jenkins Plugins

Jenkins plugins are extensions that enhance Jenkins' functionality.

Plugin Installation (in GUI)

There are two ways to install plugins in GUI:

1. Automatic Installation (.jpi - Jenkins Plugins)

- Login to Jenkins GUI.
- Go to **Dashboard → Manage Jenkins → Plugins**.
- Select **Available Plugins**.
- Search for the required plugin.
- Select the checkbox → Click **Install**.
- Tick the checkbox to **Restart Jenkins**.

2. Manual Installation (.hpi - Hudson Plugins)

- Download plugins from: In Two ways we can download
  - Official Jenkins Plugins
  - **Jenkins GUI → Available Plugins**
- Choose the required version and download.
- Upload the `.hpi` file manually in Jenkins:
  - **Dashboard → Manage Jenkins → Plugins → Advanced Settings → Upload Plugin → Deploy**.
- Check the **Deployment Progress** and restart if needed.

Uninstall Plugins

- **Dashboard → Manage Jenkins → Plugins**.
- Select the **(X)** option next to the plugin.
- Click **YES** to confirm.

Update Plugins

- **Dashboard → Manage Jenkins → Plugins → Updates**.
- Tick the checkboxes for plugins to update.
- Click **Update** (next to the search box).
- Restart Jenkins after updating.

A **Job/Project** in Jenkins consists of tasks like building, testing, and deploying code.

Types of Jenkins Projects

- **Maven Project –** Automates builds using POM files.
- **Pipeline –** Manages workflows across multiple build agents.
- **Multi-configuration Project –** Supports multiple environments/configurations.
- **Folder –** Groups items under separate namespaces.
- **Multibranch Pipeline –** Creates pipelines for each SCM branch.
- **Organization Folder –** Scans repositories to create multibranch subfolders.

overview Job/Project

Steps:

1. **Dashboard** → **New Item** → **Enter Job Name**.
2. Select an item type (**e.g., Freestyle Project**) → Click **OK**.
3. Configure the job:
   - **General** – Add project description.
   - **Source Code Management –** Add repository URLs.
   - **Build Triggers –** Define automated build conditions.
   - **Build Environment –** Set up pre-build configurations.
   - **Build Steps –** Define tasks like compiling and testing.
   - **Post-Build Actions –** Add actions like artifact archiving or notifications.
4. Click **Save**.

Freestyle Job/Project

1. **Dashboard** → **New Item** → **Enter Job Name**.
2. Select **Freestyle Project** → Click **OK**.
3. In the configuration, go to **Build Steps** → Click **Add Build Step**.
4. Select **Execute Windows Batch Command**.
5. Type `dir` → Click **Save**.
6. Go to **Dashboard**, select the job.
7. Click **Build Now**.

Jenkins Folder Structure

In **Windows OS**, Jenkins stores all its information and configuration in the following path:

```
C:\ProgramData\Jenkins\.jenkins\
```

Folder Breakdown:

1. `workspace` :
   - Contains all the information about **configured jobs**.
   - Stores the workspace and build artifacts for each job.
2. `secrets` :
   - Stores **configured secrets** such as credentials and tokens.
   - Ensures sensitive data is handled securely.
3. `plugins` :
   - Contains all **installed plugins**, both manually installed and automatically by Jenkins.
   - Each plugin has its own directory, including configuration files and resources.
4. `nodes` :
   - Contains information about **configured nodes** (master or agents).
   - Stores node configuration details.
5. `logs` :
   - Stores **log files** related to Jenkins operations.
   - Includes logs from agents (if configured) and task logs.
   - You can also access Jenkins logs through the **Jenkins GUI**:
     - **Manage Jenkins → System Log → All Jenkins Logs**.
     - The logs provide detailed information on Jenkins system activities, job executions, user authentication, plugin activity, and internal Jenkins events.

Important Jenkins Configuration File:

- `config.xml` :
  - The **primary configuration file** for Jenkins.
  - Located in `JENKINS_HOME/config.xml` .

- Contains key configurations related to global settings, security, and plugin configurations.
- **Important**: Always **backup** the `config.xml` file before modifying it.

What If a Jenkins User Forgets Their Password?

1. Resetting the Password (Admin Access Required)

Via Jenkins GUI (for Admins):

- Navigate to **Manage Jenkins** → **Security** → **Users**.
- Find and select the user whose password needs to be reset.
- In the **Security** section, reset the password:
  - Enter the new password and confirm it.
  - Click **Save** to apply the changes.

User Must Reset Their Own Password:

- After the admin resets the password, the user must reset it themselves:

Steps for the User:

- Log in to Jenkins.
- Click on the **username** at the top right corner.
- Go to **Security**.
- Enter and confirm the **new password**.
- Click **Save**.

2. File-Based Password Reset (Not Recommended)

- **Location of User Configuration File**:
  - User configurations are stored in:
    `JENKINS_HOME/users/<user_name>/config.xml`

  Warning:
  - **Modifying** the `config.xml` file directly is **not recommended** as it can corrupt the user configuration.
  - Instead, you can **delete the user's folder**, but this will result in the loss of their settings and data.

3. External Authentication Systems (LDAP/Active Directory)

- If Jenkins is using **external authentication** (e.g., **LDAP** or **Active Directory**), passwords are not stored in Jenkins.
- Users must reset their password via the external authentication provider (e.g., AD/LDAP system).
- Contact the system administrator for assistance with password resets in the external system.

4. Deleting and Re-Creating the User (Last Resort)

- If the normal user cannot be reset due to limited access or external authentication, you could:
- Delete the user (as an admin).
- Recreate the user: Admin can create a new user with the same username, ensuring the user can log in again (though this may cause loss of previous settings for the user).

What To Do If Jenkins Admin Forgets Their Password

If you forget your Jenkins admin password, follow these steps to regain access.

1. Backup Jenkins Configuration

- Navigate to Jenkins Home Directory:
  - Path: `C:\ProgramData\Jenkins\.jenkins\`
- Take a backup of the `config.xml` file in case anything goes wrong during the process.

2. Disable Security Temporarily

- Open the `config.xml` file in a text editor.
- Locate the following line:

```
<useSecurity>true</useSecurity>
```

- Change the value from `true` to `false`:

```
<useSecurity>false</useSecurity>
```

- Save the changes to `config.xml`.

3. Restart Jenkins

- Restart Jenkins to apply the changes.
- Access Jenkins via the web browser. You should be able to log in without authentication.

4. Modify Security Settings

- In Jenkins, go to **Manage Jenkins**.
- Click on **Security** under the Security section.

  Security Realm:

  - Change the **Security Realm** from `None` to **Jenkins' own user database**.
  - Save the changes.

  Authorization:

  - Change the **Authorization Strategy** from **Anyone can do anything** to either **Matrix-based security** or **Project-based Matrix Authorization Strategy**.
  - Click **Save**.

5. Reset password Admin User

- In Jenkins, go to **Manage Jenkins**, go to **users**
- Select the admin you want to reset.
- go to **security**.
- change password and **save** it.

6. Change Admin User Password

- Dashboard -- Manage Jenkins -- Security
- under **Authorization Strategy**.
- click **add user**.
- add the **admin** and give admin privileges.
- Uncheck **Anonymous user** to remove administrative privileges from anonymous users.
- Click **Save**.

7. Restore Security Settings

- After restarting Jenkins, verify that the newly created admin user has full administrative privileges.
- Return to the `config.xml` file. (check it changes to true)
- Change the `<useSecurity>` setting back to `true` :

```
<useSecurity>true</useSecurity>
```

- Save the file.

8. Final Restart

- Restart Jenkins one more time to finalize all changes.
- Confirm that the system is secure, and the admin user has the proper permissions.

These steps should help restore administrative access to Jenkins while keeping the system secure.

How to Change the Default Port Number of Jenkins

If you need to change the default port number Jenkins uses (8080), follow these steps:

- Go to the Jenkins installation directory:
  - Path: `C:\Program Files\Jenkins`
- Before making any changes, **take a backup** of the `jenkins.xml` file to ensure you can restore the original settings if needed.
- Open the `jenkins.xml` file in a text editor (e.g., Notepad++ or Visual Studio Code).
- Search for the following line in the file:

```
--httpPort=8080
```

- Replace `8080` with your desired port number, for example:

```
--httpPort=9090
```

- After modifying the port number, save the `jenkins.xml` file.
- Restart Jenkins for the changes to take effect.
- After Jenkins restarts, it will be accessible on the new port you specified (e.g., `http://localhost:9090`).

Ways to Trigger Jenkins Jobs

1. Trigger Builds Remotely

Allows triggering Jenkins builds remotely via a predefined URL, which is useful for scripts or external systems like SCM hooks.

Steps to Create an API Token:

- Click on your **username/admin name** at the top right corner.
- Go to **Security**.
- Under **API Token**, click **Add New Token**.
- Copy the token (you can only view it once during creation).
- Click **Save**.

Usage:
You can trigger a build remotely using the following URL:

```
JENKINS_URL/job/<job_name>/build?token=<TOKEN_NAME>
```

Alternatively, for builds with parameters:

```
JENKINS_URL/job/<job_name>/buildWithParameters?token=<TOKEN_NAME>
```

1.3 Build Trigger Automatically

A real-life scenario could be triggering jobs automatically, such as after a deployment is completed.

- **Trigger Jobs Remotely with Scripts**:
  - You can use the **remote trigger** mechanism with URLs and tokens to trigger jobs programmatically. This is useful for scenarios like:
    - Testing teams triggering jobs after deployments.
  - Configure the job with an **Authentication Token**.
  - Remote teams can then use the token to trigger the job remotely via scripts or webhooks.

Triggering Jobs Remotely in Jenkins

1. **Triggering a Job Using Authentication Token:**
   - In a **Freestyle Job**:
     a. Create a new Freestyle job, e.g., **Build Remotely**.
     b. Check the **Trigger builds remotely** option.
     c. Configure an **Authentication Token** (e.g., `remote-123`).
   - **URL Syntax** to trigger the job remotely:

        - `JENKINS_URL/job/job_name/build?token=TOKEN_NAME`
        - Example: `http://localhost:9999/job/basicpipeline/build?token=remote-123`
    - Here, `TOKEN_NAME` is the authentication token you configured earlier.

2. **Triggering a Job Using `curl` Command:**
    - You can also use the following **curl** command to trigger the job remotely:

```
curl -u user:API_TOKEN -X POST http://localhost:8999/job/job_name/build
```

    - **Explanation of the command:**
        - `-u user:API_TOKEN` : User credentials with an API token.
        - `-X POST` : Specifies the HTTP POST method to trigger the build.
        - `http://localhost:8999/job/job_name/build` : The URL of the Jenkins job you want to trigger.
        - **API_TOKEN** should be generated by the **admin user**.

This structure makes it easier to follow and understand how to configure tools and trigger jobs remotely in Jenkins.

2. Build After Other Projects are Built (Upstream and Downstream)

Configure a job to trigger after other projects are built. This is useful for running tests after a build, for example.

- **Steps**:
    - In the job configuration, under **Build Triggers**, select **Build after other projects are built**.
    - Specify the **job name** of the project to watch.
    - Choose one of the options:
        - **Trigger only if the build is stable**
        - **Trigger even if the build is unstable**
        - **Trigger even if the build fails**
        - **Always trigger, even if the build is aborted**

Build After Other Projects

You can configure Jenkins to trigger a job after another job completes.

- **Create Multiple Jobs**:
  - For example, create three jobs: **Job1**, **Job2**, and **Job3**.
- **Configure Downstream Jobs**:
  - For **Job1**, configure it to trigger **Job2** under the **Build Triggers** section. (Job2 is the downstream job).
  - For **Job2**, configure it to trigger **Job3** similarly.
- **Result**:
  - When **Job1** is triggered, it will automatically trigger **Job2**, and once **Job2** finishes, it will trigger **Job3**.

3. Build Periodically

Configure Jenkins to trigger a build at regular intervals using **cron syntax**.

Cron Syntax:

```
MINUTE HOUR DOM MONTH DOW
```

- **MINUTE**: 0–59 (minute of the hour)
- **HOUR**: 0–23 (hour of the day)
- **DOM**: 1–31 (day of the month)
- **MONTH**: 1–12 (month)
- **DOW**: 0–6 (day of the week, where 0=Sunday)

Example:

- `H/5 * * * *` (Polls every 5 minutes)
- `H 4 * * 1-5` (Polls at 4 AM, Monday to Friday)

Build Periodically / Poll SCM Trigger

To trigger a Jenkins job periodically or based on changes in source control, follow these steps:

- **Create a New Job**:

- Go to Jenkins dashboard and click on **New Item**.
- Select **Freestyle project** and name your job.

- **Configure Build Periodically**:
  - Go to **Configure** of your job.
  - Under **Build Triggers**, check the box for **Build Periodically**.
  - Under **Schedule**, configure the schedule using cron syntax. Example:

    ```
    * * 8 2 6
    ```

    Example: `* * 8 2 6` will trigger the job every minute, every hour, on Saturday, 8th February.

4. GitHub Hook Trigger for Git SCM Polling

Trigger builds when Jenkins receives a GitHub push hook. Jenkins checks whether the hook came from the GitHub repository defined in the **SCM/Git** section of the job.

Note: This requires the **GitHub plugin** to be installed and the GitHub webhook to be configured.

5. Poll SCM

- Jenkins checks for changes in the source code repository at regular intervals. If changes are detected, Jenkins triggers a build.
- The polling schedule is configured using **cron syntax**.
- **Advantages**: Ideal for continuous integration, but can be **resource-intensive** and may cause **delays** depending on the polling interval.

1. Pipeline Projects Overview

1.1 Why Use Pipeline Projects?

Pipeline projects are ideal for automating and securing CI/CD pipeline code. They allow you to store pipeline definitions in a version control system (SCM) and provide better scalability and flexibility for complex workflows.

1.2 Creating a Simple "Hello World" Pipeline Job

1. **Create a New Pipeline Job**:
    - Click on **New Item**.
    - Select **Pipeline** and click **OK**.
2. **Configure the Pipeline**:
    - In the job configuration page, scroll to **Pipeline**.
    - You can write the pipeline script directly in the **Script** text area or pull it from SCM.
3. **Example: "Hello World" Pipeline Script**: You can write a simple "Hello World" pipeline script like this:

```
pipeline {
    agent any

    stages {
        stage('Hello') {
            steps {
                echo 'Hello World'
            }
        }
    }
}
```

2. Rebuild vs. Replay

2.1 Rebuild

- **Rebuild** is used when you want to trigger the same build **without changing the pipeline script** or parameters.
- It simply re-runs the job as it was executed before.

2.2 Replay

- **Replay** allows you to modify the pipeline code and then **execute the job with those changes**.
- Useful for **troubleshooting**, as you can experiment with different changes without modifying the original code.

3. Using the Snippet Generator in Jenkins Pipeline

3.1 Overview

If you're unfamiliar with how to write pipeline code, the **Snippet Generator** is a useful tool that helps you build pipeline script snippets.

- **Access Snippet Generator**:
  - In your pipeline job configuration, click on **Pipeline Syntax** in the left menu.
  - Choose **Snippet Generator**.
- **How It Works**:
  - Select a step from the list, configure it, and click **Generate Pipeline Script**.
  - The tool generates the necessary pipeline code for the selected step, which you can then copy and paste into your pipeline script.

Example: Using `echo` to Print a Message

1. **Select Echo Step**:
   - In the Snippet Generator, go to **Steps** and select **echo: print message**.
   - Enter the message you want to print.
2. **Generate the Script**:
   - Click **Generate**.
   - The generated script will look like this:

```
        echo 'Your message here'
```

- **Copy and Paste the Generated Script** into your pipeline script.

4 Pipeline Execution Mode

By default, Jenkins pipeline execution occurs **serially**:

- Each stage will run one after another.
- A stage only runs if the previous stage was successful.

If you want to run stages in parallel or control the execution flow differently, you can modify the pipeline script accordingly.

Using Environment Variables in Jenkins Pipeline

Overview:

Jenkins Pipeline exposes environment variables through the global `env` variable. These variables are accessible from anywhere within a Jenkinsfile, making it easy to access important information during the build process.

- **Link to Documentation:** Jenkins Pipeline - Using Environment Variables

Examples of Default Environment Variables:

- `BUILD_ID` : A unique identifier for the current build.
- `BUILD_NUMBER` : The number of the current build in the job's history.
- `BUILD_URL` : The URL of the current build.
- `JENKINS_URL` : The base URL of the Jenkins instance.
- `JOB_NAME` : The name of the Jenkins job.
- `WORKSPACE` : The directory where Jenkins stores the files for the current job.

Sample Script for Using Environment Variables:

Jenkinsfile (Declarative Pipeline)

```
pipeline {
    agent any
    stages {
        stage('Example') {
            steps {
                echo "Running ${env.BUILD_ID} on ${env.JENKINS_URL}"
            }
        }
    }
}
```

- This simple script echoes the `BUILD_ID` and `JENKINS_URL` environment variables during the pipeline execution.

How to Parameterize a Jenkins Job

What are Parameters?

Parameters allow you to prompt users for input when triggering a job. These inputs are passed into the build and can be used to customize the behavior of the job based on the user's selection.

Steps to Use Parameters in Jenkins Jobs:

1. **Create a New Job**:
    - Start by creating a new Jenkins job (Freestyle or Pipeline).
2. **Enable Parameterization**:
    - Go to the **General** section of the job configuration.
    - Check the box labeled "**This project is parameterized**".
3. **Add Parameters**:
    - Click "**Add Parameter**" to select from a variety of parameter types. Below are the available types:
        - **Boolean Parameter**: A true/false flag.
        - **Choice Parameter**: A dropdown with predefined options.
        - **Credentials Parameter**: Select credentials stored in Jenkins.
        - **File Parameter**: Allow the user to upload a file.
        - **Multi-line String Parameter**: For accepting multi-line input.

- **Password Parameter**: For secure password input.
- **Run Parameter**: For selecting a build to run.
- **String Parameter**: For simple text input.

4. **Example: Using a Choice Parameter**:
   - **Name**: `ENV`
   - **Choices**: Define multiple environments (e.g., `dev`, `qa`, `prod`).
   - **Description**: "Choose the environment."

   This will allow users to choose one of the environments during the build.

5. **Triggering the Job with Parameters**:
   - When the job is triggered, Jenkins will prompt for the parameter(s).
   - You will see the **Build Parameter** option, where you can select the predefined choices (like `dev`, `qa`, or `prod`) and then start the job.

Cloning a Private Git Repository in Jenkins with SSH Keys or PAT

To securely clone a private Git repository in Jenkins, you can use **SSH keys** or a **Personal Access Token (PAT)** for authentication. Below are the steps for both methods, along with a sample pipeline configuration.

Method 1: Clone Git Repository Using SSH Keys

Steps to Setup SSH Keys for Jenkins and Git (GitHub, GitLab, etc.)

1. Generate or Use Existing SSH Key Pair

If you haven't already created an SSH key pair for Jenkins, you'll need to generate one:

- On your Jenkins server or machine, run the following command to create an SSH key pair:

```
ssh-keygen -t rsa -b 4096 -C "your_email@example.com"
```

- When prompted for a file to save the key, press **Enter** to use the default path (`~/.ssh/id_rsa`).
- You can add a passphrase for added security, or leave it empty.
- This will generate two files:
  - **Private Key**: `~/.ssh/id_rsa` (keep this private and secure)
  - **Public Key**: `~/.ssh/id_rsa.pub` (this will be added to your Git service)

**2. Store SSH Public Key in Git **

GitHub:

1. **Log in to GitHub** and go to your **profile settings**.
2. Navigate to **SSH and GPG keys**:
   - Go to **Settings → SSH and GPG Keys**.
   - Click on **New SSH Key**.

3. **Paste your public key**:
    - Open the `id_rsa.pub` file (public key) on your Jenkins server using a text editor:

      ```
      cat ~/.ssh/id_rsa.pub
      ```

    - Copy the content of the file and paste it into the **Key** field on GitHub.
    - Add a descriptive title (e.g., "Jenkins SSH Key").
4. Click **Add SSH Key**.

3. Store SSH Private Key in Jenkins

Now that you have the **SSH public key** added to your Git service (GitHub, GitLab, etc.), you need to store the **private key** securely in Jenkins.

1. **Go to Jenkins Dashboard → Manage Jenkins → Manage Credentials**.
2. Select the appropriate **domain** (or **Global** if not using domains).
3. Click on **Add Credentials**.
    - **Kind**: Select **SSH Username with private key**.
    - **Username**: Enter the Git username for your Git service. For example, `git` for GitHub or GitLab.
    - **Private Key**: Choose **Enter directly** and paste the contents of your private key (`id_rsa`) here.
    - **ID**: Assign a unique ID to the credentials (e.g., `my-ssh-key`).
4. Click **OK** to save the credentials.

4. Use SSH Key in Jenkins Pipeline

- In your pipeline, use the `sshagent` block to authenticate with the SSH key.

Sample Jenkins Pipeline for SSH Authentication:

```
pipeline {
    agent any

    environment {
        // Define the SSH credentials ID and Git repository URL
        SSH_CREDENTIALS_ID = 'my-ssh-key'
        GIT_REPO_URL = 'git@github.com:yourusername/your-private-repo.git'
    }

    stages {
        stage('Clone Repository') {
            steps {
```

```
                script {
                    // Use SSH authentication to clone the private repository
                    sshagent([SSH_CREDENTIALS_ID]) {
                        sh 'git clone ${GIT_REPO_URL}'
                    }
                }
            }
        }
        // Add other stages like build, test, deploy, etc.
    }
}
```

Method 2: Clone Git Repository Using Personal Access Token (PAT)

Steps to Setup Personal Access Token (PAT):

1. **Generate a PAT on GitHub (or other Git services):**
   - Go to your **GitHub profile** → **Settings** → **Developer Settings** → **Personal Access Tokens**.
   - Click **Generate new token** and provide the necessary **permissions** (e.g., `repo` for full control over private repositories).
   - **Save the token** somewhere safe, as you won't be able to view it again once generated.
2. **Store PAT in Jenkins:**
   - Go to **Jenkins Dashboard** → **Manage Jenkins** → **Manage Credentials**.
   - Choose the **Global** or a specific **Domain** for the credentials.
   - Click on **Add Credentials**.
     - **Kind**: Select **Username with password**.
     - **Username**: Enter your GitHub username (or the username of your Git provider).
     - **Password**: Paste your **Personal Access Token**.
     - **ID**: Assign a unique **ID** to the credentials (e.g., `github-pat`).
3. **Use PAT in Jenkins Pipeline:**
   - In your pipeline, reference the PAT credentials to authenticate when cloning the repository.

Sample Jenkins Pipeline for PAT Authentication:

```
pipeline {
    agent any
```

```
    environment {
        // Define the credentials ID and Git repository URL
        GIT_CREDENTIALS_ID = 'github-pat'
        GIT_REPO_URL = 'https://github.com/yourusername/your-private-repo.git'
    }

    stages {
        stage('Clone Repository') {
            steps {
                script {
                    // Clone the private repository using PAT for authentication
                    checkout([
                        $class: 'GitSCM',
                        branches: [[name: '*/main']],  // Adjust the branch as needed
                        userRemoteConfigs: [[
                            url: "${GIT_REPO_URL}",
                            credentialsId: "${GIT_CREDENTIALS_ID}"
                        ]]
                    ])
                }
            }
        }
        // Additional stages (e.g., build, test, deploy) can follow
    }
}
```

Key Considerations

- **SSH Key Authentication** is recommended for security, as it doesn't expose your credentials in the URL.
- **Personal Access Token (PAT)** is an alternative when SSH keys aren't feasible. PATs can be more flexible for automated systems and can be scoped to specific permissions.
- Always **secure your credentials** by storing them in Jenkins' **Credentials** store, and **never hard-code them** in pipeline scripts.

Troubleshooting

- **"Permission denied" error**: If you encounter an error such as `Permission denied (publickey)`, ensure that:

    - The **public key** has been added to your Git service (e.g., GitHub) under **SSH Keys** in your account settings.
    - Jenkins is using the correct **SSH key** for authentication.

- "**Authentication failed" with PAT**: If using a PAT and the clone fails:
    - Ensure that the **PAT** has the necessary permissions (e.g., `repo`).
    - Verify the **credentials** are correctly configured in Jenkins.
    - Check if the URL format for the repository is correct (use HTTPS for PAT).

Summary:

- **SSH Keys** provide more secure authentication for cloning repositories.
- **PAT** is another method, often simpler for setups where SSH is difficult to configure.
- Always store sensitive credentials (like SSH keys or PATs) in Jenkins' **Credentials Manager** and reference them securely in your pipeline code.

Certainly! Below is an improved structure of your notes, formatted with headlines, bullet points, and slight improvements for clarity without removing any existing content:

Jenkins Shared Libraries

What are Shared Libraries?

- A **library** is a collection of reusable code, such as functions and classes, that can be utilized across multiple Jenkins pipelines.
- The goal is to **avoid repetition** and improve maintainability by centralizing common functionality into libraries.
- Shared libraries in jenkins is a collection of grrovy scripts shared between different jenkins jobs.
- Shared libraries are stored in git repositories.
- Creating a Shared libraries simplifies the process of pushing source code updates for a project.
- Updating the library source code also updates the code of every project, that uses library.

Types of Jenkins Shared Libraries

Jenkins supports two types of shared libraries:

1. **Global Trusted Pipeline Libraries**
    - These libraries are available to any pipeline job running on this system.
    - **Trusted**: They run without sandbox restrictions, allowing the use of advanced features like `@Grab`.
2. **Global Untrusted Pipeline Libraries**
    - These libraries are also available to any pipeline job running on this system.
    - **Untrusted**: They run with sandbox restrictions, meaning they cannot use certain features like `@Grab`.

Configuring Shared Libraries in Jenkins

To set up shared libraries, follow these steps:

1. **Go to Jenkins Dashboard**
    - Navigate to **Manage Jenkins** → **Configure System**.
2. **Add Shared Library**
    - Under the **Global Pipeline Libraries** section, add a new library.
    - Provide the **Git repository URL** of the shared library. For example: `https://github.com/xaravind/shared_lib.git`
    - **Credentials**: If it's a private repository, make sure to provide the necessary credentials.

Shared Library Folder Structure Example

Here's an example of the directory structure for a shared library that includes reusable variables (`vars`) with `.groovy` files:

```
shared-lib/
├── vars/
│   ├── checkProc.groovy
│   └── helloWorld.groovy
```

- **`vars/`** : This folder contains simple Groovy scripts that define reusable functions or variables.
- **`checkProc.groovy`** and **`helloWorld.groovy`** are two examples of reusable scripts.

Example Code Using Shared Libraries

Here is a sample Jenkinsfile that demonstrates how to use a shared library:

```
@Library('my-shared-library') _  // Load the shared library

pipeline {
    agent any

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Build') {
            steps {
                script {
                    // Call the helloWorld function from shared library
                    helloWorld()
                }
            }
        }

        stage('Deploy') {
            steps {
                script {
                    // Call the checkProc function from shared library
                    checkProc()
                }
            }
        }
    }

    post {
        always {
            echo 'Pipeline completed.'
        }
    }
}
```

Explanation of the Code:

- **@Library('my-shared-library') _**:
  This imports the shared library named `'my-shared-library'` into the pipeline. It makes the functions inside `vars/` accessible.

- `helloWorld()`：
  A function from the `helloWorld.groovy` file inside the `vars/` directory is called in the given stages.

- `checkProc()`：
  A function from the `checkProc.groovy` file inside the `vars/` directory is called in the given stages.

Important Notes

- When configuring shared libraries, **ensure the repository is accessible** from Jenkins, and make sure credentials are provided if it's a private repo.
- **Global Trusted Pipeline Libraries** run without sandbox restrictions, while **Global Untrusted Pipeline Libraries** are sandboxed.

Using GitHub Webhook Actions in Jenkins

Prerequisites

- Install the **Git Plugin** in Jenkins.

Configure Jenkins Job for Webhook

For Freestyle Project

1. **Create a New Job:**
    - Go to **Jenkins Dashboard** → **New Item** → Select **Freestyle Project**.
2. **Configure Source Code Management:**
    - Under **Source Code Management**, select **Git** and enter the repository URL.
3. **Enable Webhook Trigger:**
    - Under **Build Triggers**, check "**GitHub hook trigger for GITScm polling**".
    - This ensures that when Jenkins receives a GitHub push hook, the GitHub Plugin verifies if the hook matches the defined Git repository in the **SCM/Git** section of the job.

For Pipeline Project

1. **Create a New Job:**
    - Go to **Jenkins Dashboard** → **New Item** → Select **Pipeline Project**.
2. **Configure Source Code Management:**
    - Under **Pipeline** → Select **Pipeline script from SCM**.
    - Select **SCM as Git**.
    - Enter the repository URL.
    - Ensure the `Jenkinsfile` is stored in the repository.
3. **Sample Jenkinsfile Pipeline:**

```
pipeline {
    agent any

    stages {
        stage('Checkout') {
            steps {
                git branch: 'main', url: 'https://github.com/your-repo.git'
            }
        }
        stage('Build') {
            steps {
                echo 'Building the project...'
            }
        }
    }
}
```

Configure Webhook in GitHub

1. **Navigate to GitHub Webhook Settings:**
   - Go to your **GitHub repository** → **Settings** → **Webhooks**.
2. **Add a New Webhook:**
   - Click **Add Webhook**.
   - Set the **Payload URL** to:

     ```
     http://your-jenkins-url/github-webhook/
     ```

     Example:

     ```
     http://jenkins.example.com/github-webhook/
     ```

   - Set **Content Type** to `application/json`.
3. **Select Trigger Events:**
   - Recommended: **Just the push event**.
   - Alternatively, select **individual events** as needed.
4. **Save the Webhook:**

- Click **Add Webhook**.

Test the Webhook

1. Push a commit to the repository.
2. Check **Jenkins Console Output** to verify that Jenkins was triggered successfully.

Sure! Here's a more structured and improved version of your notes:

Backup/Migration of Jenkins Jobs

There are different methods to take backups of Jenkins jobs, such as:

- **Manual Backup**: Manually copying the required files.
- **Using Plugins**: Using dedicated Jenkins plugins for backup purposes.

Backup Plugins for Jenkins

Backup Plugin

The **Backup Plugin** allows you to archive and restore the Jenkins (or Hudson) home directory. This plugin provides a simple solution for backing up your Jenkins data.

Periodic Backup Plugin

- The **Periodic Backup Plugin** is an alternative to the existing backup plugin.
- It allows you to schedule periodic backups of your Jenkins data.

ThinBackup Plugin

- **ThinBackup** is a plugin that specifically backs up both global and job-specific configurations in Jenkins.
- It is a popular choice for Jenkins administrators because it is lightweight and efficient.

Installing and Configuring the ThinBackup Plugin

1. **Install the ThinBackup Plugin**
    - Go to **Manage Jenkins** → **Manage Plugins**.
    - Search for **ThinBackup** and install the plugin.
2. **Set Global Configuration**
    - Inoreder to take backup, need to configure **ThinBackup** in global settings
    - Navigate to **Manage Jenkins** → **System Configuration** → **System**.
    - Configure the following details in the **ThinBackup Configuration** section:
        - **Backup Directory**: Set the directory where backups will be stored.
        - **Scheduling**: Define how often you want the backup to run (e.g., daily, weekly).

Taking a Backup Using ThinBackup

1. Go to **Manage Jenkins** → **Tools and Actions** → **ThinBackup**.
2. Click on **Backup Now** to initiate the backup.
3. Check the backup folder in the configured backup directory to verify that the backup was successful.

Restoring a Backup

- To restore a backup:
    i. Go to **Manage Jenkins** → **Tools and Actions** → **ThinBackup**.
    ii. Click on **Restore** next to the **Backup Now** option.
    iii. Select the directory containing the backup and restore it.